

MIDI-Instruct: A Dual-Metric Benchmark for Instruction-Conditioned Symbolic Music Editing

Julia Goh
Independent researcher
juliagsy@gmail.com

October 1, 2026

Abstract

Instruction-conditioned music editing is the natural next step after text-to-audio generation, yet symbolic benchmarks still emphasize either theory question answering over score images or audio remixing without MIDI-native preservation checks. We introduce **MIDI-Instruct**, a benchmark of source MIDI, natural-language instructions, and machine-checkable edit specifications. Each item is scored on two independent axes: **EDITSUCCESS** (did the requested transformation occur?) and **PRESERVE** (were the item’s declared preservation constraints satisfied?). A companion **MIDI-Reason** track evaluates structured edit plans separately from emitted MIDI files. Version 0.1 contains 300 unique-gold items synthesized from multi-track seeds via five verifiable transforms: transpose, tempo and velocity scaling, track muting, and program change. On the 50-item held-out test split, a greedy Llama-3.2-1B plan-then-execute baseline obtains 0.44 joint pass rate, 0.467 **EDITSUCCESS**, 0.693 **PRESERVE**, and 0.20 exact plan match. It solves muting and tempo scaling but fails all program changes and velocity edits, while 28% of items produce no valid output. These results show that file validity, requested change, preservation, and plan correctness capture distinct failure modes. We release the dataset generator, programmatic grader, reference baselines, and evaluation harness.

1 Introduction

Professional music production workflows treat MIDI as an editable artifact: notes, velocities, instruments, and tempo are parameters a producer adjusts iteratively. Recent large language models can discuss music theory, transcribe audio, and generate symbolic scores, but there is no standard benchmark that asks a model to *edit* an existing MIDI file under a natural-language instruction while checking both requested changes and explicitly declared preservation constraints.

Existing work clusters into three nearby problems, none of which matches this task:

1. **Symbolic reasoning benchmarks** such as WildScore and MusicTheoryBench evaluate multiple-choice understanding over score images or theory questions, not executable edits.
2. **Audio editing systems** such as InstructME and MuseCPBench evaluate remixing and context preservation in waveform space; RefinPaint [6] performs symbolic proofreading but is not conditioned on free-text edits.
3. **Zero-shot drum editing** (Not that Groove) demonstrates instruction-driven symbolic edits with unit tests, but covers one-bar drum grooves without a preservation axis.

MIDI-Instruct fills the gap: *MIDI in, instruction in, MIDI out*, with dual metrics that mirror Music Context Preservation (MCP) but operate on symbolic features rather than rendered audio.

Contributions.

1. **MIDI-Instruct**, a JSONL benchmark format pairing MIDI, instructions, edit masks, and predicate specifications (Section 3).
2. A **dual-metric grader** reporting EDITSUCCESS, PRESERVE, joint pass rate, over-edit rate, and optional gold note-F1 (Section 4).
3. **MIDI-Reason**, a structured plan track scored independently from the output file (Section 3.3).
4. An open **generation and evaluation harness** producing 300 unique-gold pilot items from programmatic seeds, with reference and Llama-3.2-1B baselines (Sections 5–7).

Our first model evaluation reaches 0.44 joint pass rate while exact plan match is only 0.20. The large gap across operator families—perfect performance on muting and tempo scaling but no joint passes on velocity or program changes—demonstrates why aggregate output similarity alone is insufficient. Later work—text-to-MIDI generation, tool-augmented agents, and self-refining harnesses—can reuse the same item schema; only the runner changes.

2 Related work

In-the-wild symbolic reasoning. WildScore [5] sources 807 Reddit music-theory questions with score images, reporting $\approx 68\%$ accuracy for strong multimodal models. CSyMR-Bench [9] curates 126 compositional multiple-choice items from community discussions and professional examinations, and shows music21-augmented agents gain 5–7 points. These benchmarks measure *analysis*, not *transformation*.

Text-encoded symbolic understanding. ABC-Eval [12] and LilyBench [7] evaluate LLMs on ABC or LilyPond understanding and generation. Error detection is the closest task to editing, but models select errors rather than apply fixes. ChatMusician’s MusicTheoryBench [10] remains MCQ-based.

Instruction editing and preservation. Not that Groove [11] pairs drum grooves with instructions and necessary unit tests ($\approx 68\%$ pass rate for GPT-4.1). MuseCPBench [8] unifies MCP evaluation across harmony, rhythm, form, and motifs, primarily on audio outputs. RefinPaint [6] performs iterative symbolic inpainting and proofreading, but is not conditioned on free-text edit instructions. BeatEdit [2] shows explicit MIDI editing beats autoregressive regeneration when gold diffs exist, but does not use natural language.

Audio instruction following. CMI-Bench [4] reframes MIR tasks as instructions over audio. InstructME [3] edits mixed audio with latent diffusion. These systems inform metric design but do not evaluate MIDI-native dual axes.

Positioning. To our knowledge, MIDI-Instruct is the first open benchmark combining (i) multi-track MIDI input and output, (ii) natural-language instructions, (iii) programmatic edit and preserve predicates, and (iv) a separate plan track for reasoning evaluation.

3 Benchmark design

3.1 Task definition

Given source MIDI x , instruction e , a system produces edited MIDI $\hat{x}$ and optionally a structured plan $\hat{p}$. The instruction specifies attributes that *must change*; all other attributes *should preserve* unless the instruction explicitly overrides them. The current grader operationalizes this goal through an explicit, finite list of preservation predicates per item rather than claiming exhaustive equivalence over every MIDI event type.

Each benchmark item stores:

- paths to x and (for unique gold) reference x^* ;
- instruction e and operator family label;
- edit mask m (tracks and optional time window);
- lists of `must_change` and `must_preserve` predicates;
- optional gold plan p for MIDI-Reason.

3.2 Gold modes

Unique gold (v0.1). Apply deterministic transform T_θ to seed MIDI, verbalize θ as e , store $x^* = T_\theta(x)$. Enables note-F1 and exact plan matching.

Constraint gold (planned). For descriptive instructions with many valid outputs, store necessary unit tests (Not that Groove style) without a single x^* . Deferred until unique-gold grader validation completes.

3.3 MIDI-Reason track

The plan $p = (\text{op}, \theta)$ is scored with exact match independently from $\hat{x}$. This separates *naming* the edit from *executing* it—the same distinction agent security benchmarks draw between belief probes and tool-call behavior.

3.4 Operator families (v0.1)

Transpose (non-drum tracks), tempo scale, velocity scale, track mute, and program change. All operators are programmatically verifiable; no LLM judge is used for headline metrics.

4 Metrics

Let P_c and P_p be the must-change and must-preserve predicate sets. Each predicate returns a score in $[0, 1]$.

$$\text{EditSuccess} = \frac{1}{|P_c|} \sum_{p \in P_c} \text{score}(p; x, \hat{x}) \quad (1)$$

$$\text{Preserve} = \frac{1}{|P_p|} \sum_{p \in P_p} \text{score}(p; x, \hat{x}) \quad (2)$$

$$\text{Joint} = \mathbb{K}[\text{EditSuccess} \geq \tau \wedge \text{Preserve} \geq \tau] \quad (3)$$

with default $\tau = 0.9$.

Additional reports.

- **Validity:** binary MIDI parse check.
- **OverEdit:** symmetric-difference rate on notes outside edit mask m , capped at 1.
- **Gold note-F1:** for unique gold, F1 between $\hat{x}$ and x^* inside m .
- **Plan match:** exact equality on (op, θ) for MIDI-Reason.

Predicate examples. Pitch shift checks median pitch delta on masked tracks with greedy onset matching (supports chords and flams); track mute requires peak velocity at or below a threshold (default 1) while keeping note timing and pitch fixed; tempo scale compares end-time duration ratio; preservation uses exact note-set equality outside the mask, pitch histogram JS divergence, and track-set identity. Tempo edits use a global `pitches_unchanged` predicate because timing changes affect all note onsets.

Human listening tests may calibrate descriptive items later; they are not headline metrics in v0.1.

5 Dataset construction

5.1 Seeds

Version 0.1 uses 12 programmatic multi-track seeds (piano, bass, drums; four bars each) generated by the toolkit and distributed under the repository license. Each seed defines a `composition_id`; all derived items inherit the same split assignment to prevent leakage.

5.2 Splits

Seeds 0–7 $\rightarrow$ train, 8–9 $\rightarrow$ validation, 10–11 $\rightarrow$ test. Future releases may add licensed corpora (Mutopia, Lakh subsets) with the same split policy.

5.3 Scale

The generator emits items from the Cartesian product of seeds, operator parameters, and instruction templates. Full generation yields 336 items: 192 transpose (8 semitone steps $\times$ 2 phrasings), 36 velocity, 36 tempo, 24 mute, and 48 program change. The released pilot trims proportionally within each composition so every seed remains represented. Its 300 items comprise 168 transpose, 24 velocity, 36 tempo, 24 mute, and 48 program-change examples, split into 200 train, 50 validation, and 50 test items.

5.4 Reproducibility

```
musicinstruct generate-pilot --output-dir data/pilot --target 300
musicinstruct validate data/pilot/pilot.jsonl
musicinstruct self-test data/pilot/pilot.jsonl
```

Self-test scores gold MIDI against itself; joint pass rate should be 1.0, validating the grader before model experiments.

6 Experimental setup

Reference baselines. We run four programmatic baselines on the test split (**run-baselines**): **oracle** emits gold MIDI and plan; **copy_source** returns the unchanged input; **wrong_transform** executes the correct operator family with deliberately incorrect parameters; and **no_output** represents total generation failure. These controls test the expected metric boundaries rather than serve as competitive systems.

Plan-then-execute baseline. We evaluate `meta-llama/Llama-3.2-1B-Instruct` [1] as an open, lightweight instruction-following baseline. The model receives the natural-language edit instruction, allowed operation schema, and a summary of available MIDI tracks, then greedily generates a JSON plan with temperature 0 and a 128-token output cap. A deterministic executor validates the plan and applies the requested operation using `pretty_midi`; malformed plans or execution errors receive no MIDI output and are retained as failed predictions. Inference uses half precision on a single NVIDIA T4. No examples from the train split, fine-tuning, retries, or output repair are used.

Prediction format. Models write one MIDI file per item. JSONL predictions reference `midi_path` relative to the predictions file and optional `plan`.

Reporting. We report mean EDITSUCCESS, PRESERVE, joint pass rate, OverEdit, and gold note-F1 overall and by `op_family`. Plan match is reported separately as MIDI-Reason. Validity counts whether the runner emits a parseable MIDI; prediction coverage counts submitted JSONL records and is 100% because failed attempts remain explicit records. The held-out test set contains 50 items from two compositions (`seed_10` and `seed_11`): 28 transpose, 8 program-change, 6 tempo, 4 velocity, and 4 mute items.

System	Valid	EDITSUCCESS	PRESERVE	Joint	OverEdit	Plan
Oracle	1.000	1.000	1.000	1.000	0.000	1.000
Copy source	1.000	0.112	1.000	0.000	0.000	0.000
Wrong transform	1.000	0.332	0.973	0.000	0.061	0.000
No output	0.000	0.000	0.000	0.000	1.000	0.000
Llama-3.2-1B	0.720	0.467	0.693	0.440	0.320	0.200

Table 1: Test results on 50 items. “Plan” is exact structured-plan match. Gold note-F1 equals 1.000 for the oracle, 0.160 for copy source, 0.161 for wrong transform, 0 for no output, and 0.440 for Llama-3.2-1B.

Grader validation. Before publishing model numbers, run `self-test` on the released manifest. Any joint < 1 indicates predicate or transform bugs, not model failure. `stress-test` applies transpose and tempo transforms to external MIDIs (local directories or MidiCaps/Lakh paths) and self-scores gold vs. gold to catch parser edge cases on real files.

Reproducibility. The evaluation harness writes all 50 prediction records, generated MIDI files, aggregate reports, and per-item predicate details; these files must be kept together because prediction paths are relative. The same CLI commands regenerate programmatic baselines and rescore model outputs. We will archive the complete result bundle with the paper release rather than relying on machine-specific paths in intermediate reports. Constraint-gold descriptive items, licensed seed corpora, multi-step instructions, tool-augmented agents, and self-refinement can reuse the schema with additional runners.

7 Results

7.1 Overall comparison

Table 1 reports test-set results. The oracle reaches a joint pass rate of 1.0, confirming that the generated references satisfy every edit and preservation predicate. Neither copying the source nor applying a deliberately wrong transform passes any item jointly, even though both produce valid MIDI and satisfy most declared preservation predicates. This separation is intentional: preservation alone is insufficient when the requested edit is absent or incorrect.

The Llama plan-then-execute baseline obtains a joint pass rate of 0.44 (22 of 50 items). Its mean EDITSUCCESS is 0.467 and mean PRESERVE is 0.693. Only 72% of its predictions yield an executable, parseable MIDI file; failed plan generation is scored as an invalid output rather than omitted. Exact plan match is 0.20, showing that successful file-level edits do not imply exact recovery of the benchmark’s canonical plan parameters.

7.2 Performance by operation

Performance varies sharply by operator family (Table 2). Track muting and tempo scaling are solved perfectly, including exact plans. Transposition reaches 0.429 joint: all completed positive shifts of +2, +5, and +7 semitones pass, while negative instructions are emitted as positive shifts and all ± 3 cases fail during plan generation. Velocity scaling preserves the source but does not make

Operation	<i>n</i>	Valid	EDITSUCCESS	PRESERVE	Joint	Plan
Mute tracks	4	1.000	1.000	1.000	1.000	1.000
Tempo scale	6	1.000	1.000	1.000	1.000	1.000
Transpose	28	0.714	0.429	0.714	0.429	0.000
Velocity scale	4	1.000	0.333	1.000	0.000	0.000
Program change	8	0.250	0.000	0.083	0.000	0.000

Table 2: Llama-3.2-1B results by operator family. Values are macro-averages over items; the joint column is a pass rate.

Item	Error	Model output (truncated)
seed_10_program_25	out-of-range track	{"op": "transpose", "semitones": 5, "tracks": [0,2,3]}
seed_10_transpose_3	incomplete plan	{"op": "transpose", "semitones": 3, "tracks": [0..35, ...]}
seed_10_transpose_2	sign reversal	{"op": "transpose", "semitones": 2, ...} (expected -2)
seed_10_velocity_0	wrong operator	{"op": "transpose", "semitones": 0, ...}
seed_11_program_48	wrong parameter	program=0 (expected 48)

Table 3: Representative Llama-3.2-1B failure modes on the test split.

the requested change, resulting in partial predicate credit but no joint passes. Program changes are the hardest family: none passes, and six of eight do not produce a valid output.

7.3 Failure analysis

Fourteen predictions produce no MIDI: 11 responses do not contain a complete parseable plan under the 128-token generation cap, and three reference an out-of-range track. The incomplete responses often begin an excessively long track list, indicating a formatting failure rather than a MIDI parser failure. Among the 36 executable outputs, the main semantic errors are sign reversal for downward transposition, mapping velocity scaling to a zero-semitone transpose, and interpreting General MIDI program numbers as transpose intervals. These errors expose three distinct bottlenecks that a single accuracy number would merge: structured-plan formation, safe execution, and preservation of unrelated content.

The comparison also illustrates why both axes are needed. Copy source attains perfect PRESERVE but zero joint, while the model’s invalid or misdirected edits lower preservation to 0.693 and raise OverEdit to 0.320. Copy source nevertheless receives gold note-F1 of 0.16 because note events are unchanged by program-change references; the dedicated `program_is` predicate correctly assigns zero edit success. Thus note-F1 is useful only as a supplementary, operation-dependent diagnostic. Conversely, strict plan match can be lower than file-level success: 12 of 28 transpose items pass jointly but have plan match 0.0 because the model lists drum track 2 in `tracks` while gold plans target only non-drum tracks [0,1]. Execution still satisfies pitch-shift predicates because drums are masked. File-level predicates therefore remain the primary task metric, with MIDI-Reason reported as a diagnostic track.

8 Limitations

- **Synthetic seeds.** v0.1 items are programmatic, not drawn from professional arrangements; absolute scores may not transfer to in-the-wild MIDI complexity.
- **Small held-out composition set.** The 50 test items derive from only two seed compositions. Multiple parameterizations of the same seed are correlated, so item-level averages should not be interpreted as confidence across musical styles.
- **Unique gold only.** Descriptive instructions with many valid outputs are deferred; current metrics may over-penalize creative but valid edits once constraint gold arrives.
- **Predicate scope.** PRESERVE evaluates each item’s declared predicate list, not every possible MIDI event or metadata field. In particular, within-mask attributes not named by a predicate may change without reducing PRESERVE. Broader releases should add operation-specific invariants and adversarial tests.
- **Operator coverage.** Five programmatically verifiable families do not cover harmonization, re-orchestration, or bar-level rewriting; difficulty is intentionally controlled for grader validation.
- **Single model baseline.** The model study uses one small open LLM with greedy decoding and no retries. It establishes that the harness exposes interpretable failures, not a ranking of current instruction-following systems.
- **Plan representation.** Exact plan match can reject a plan that produces a predicate-correct file when parameters differ from the canonical representation. We therefore treat plan match as diagnostic and file-level joint success as primary.
- **Generation cap.** Eleven failures are incomplete plans under a 128-token cap. A larger cap, constrained decoding, or schema repair would likely improve validity, so the reported model score characterizes this fixed runner rather than the model in isolation.
- **Tempo estimation.** BPM ratio checks depend on `pretty_midi` tempo estimation; extreme rubato seeds may add noise.

9 Conclusion

MIDI-Instruct defines instruction-conditioned symbolic music editing as a benchmarkable task with dual edit and preserve metrics, a separate MIDI-Reason plan track, and an open synthetic dataset generator. The oracle’s 1.0 joint score and the zero joint scores of copy-source and wrong-transform controls validate the intended metric boundaries. A Llama-3.2-1B plan-then-execute baseline reaches 0.44 joint success, with perfect muting and tempo edits but systematic failures on velocity scaling, program changes, negative transposition, and structured output validity. These operator-level differences would be obscured by validity, preservation, or plan accuracy alone. By grading emitted MIDI with programmatic predicates rather than LLM judges and retaining failed attempts in the denominator, the benchmark supports reproducible diagnosis of future text-to-MIDI systems, editing models, and tool-augmented agents. The next release should broaden composition diversity, add constraint-gold instructions, and compare constrained decoding and tool-augmented runners.

References

- [1] Abhimanyu Dubey et al. The Llama 3 herd of models. *arXiv preprint arXiv:2407.21783*, 2024.
- [2] Haoyu Gu et al. Beatedit: Symbolic music generation as explicit editing. 2026.
- [3] Ao Lv et al. InstructME: An instruction guided music edit framework with latent diffusion models. In *IJCAI*, 2024.
- [4] Ying Ma et al. CMI-Bench: A comprehensive benchmark for evaluating music instruction following. In *ISMIR*, 2025.
- [5] Gagan Mundada, Yash Vishe, Amit Namburi, Xin Xu, Zachary Novack, Julian McAuley, and Junda Wu. Wildscore: Benchmarking MLLMs in-the-wild symbolic music reasoning. In *EMNLP*, 2025.
- [6] Pedro Ramoneda, Martín Rocamora, and Taketo Akama. Music proofreading with RefinPaint: Where and how to modify compositions given context. In *Proceedings of the 25th International Society for Music Information Retrieval Conference*, pages 240–247, 2024.
- [7] Matteo Spanio, Mohammad Torabi, Andrea Poltronieri, and Antonio Rodà. Can LLMs understand LilyPond? a benchmark for symbolic music generation and understanding. *arXiv preprint arXiv:2606.08722*, 2026.
- [8] Yash Vishe et al. MuseCPbench: An empirical study of music editing methods through music context preservation. 2025.
- [9] Boyang Wang, Yash Vishe, Xin Xu, Zachary Novack, Xunyi Jiang, Julian McAuley, and Junda Wu. CSyMR: Benchmarking compositional music information retrieval in symbolic music reasoning. *arXiv preprint arXiv:2601.11556*, 2026.
- [10] Ruibin Yuan et al. Chatmusician: Understanding and generating music intrinsically with LLMs. In *ACL Findings*, 2024.
- [11] Li Zhang. Not that groove: Zero-shot symbolic music editing. *arXiv preprint arXiv:2505.08203*, 2025.
- [12] Jiahao Zhao, Yunjia Li, Wei Li, and Kazuyoshi Yoshii. ABC-Eval: Benchmarking large language models on symbolic music understanding and instruction following. 2025.